Effective Testing

COMSM0086

Dr Simon Lock & Dr Sion Hannuna

Testing

In OTHER units you learn the THEORY of testing

In THIS unit you get chance to apply it in PRACTICE

It is only HERE you really appreciate the challenge !

Perhaps some advice to improve your testing ?

Importance of Testing

Typically, developers don't *enjoy* writing tests
But it's *essential* part of professional development

With gen AI, testing becomes EVEN more important !
AI tools generate large amounts of code VERY quickly
But we can't trust that code without extensive testing

We've already encountered the JUnit test framework
It would be good to think about how BEST to use it

Be Systematic

It is essential to be systematic when writing tests:

- Consider ALL required features
- Consider ALL variants of those features

- Consider what the system SHOULD do
- Consider what the system should NOT do

- Identify edge cases (where behaviour is strange)
- Identify corner cases (where edges intersect)

- Consider different levels of abstraction...

Levels of Abstraction

Devise your tests to target multiple levels:

Element testing: Exercise each & every line of code
For example, make sure ALL branches are executed

Unit Testing: Fully test operation of each method
Test different combinations of various parameters

Integration Testing: Test all high-level features
Ensure all user goals can be achieved using system

Equivalence Partitions

The fundamental principle of Equivalence Partitions:
We don't need to test EVERY SINGLE input value
We can CLUSTER input values together by similarity

Pick representative test values from each "cluster"
This drastically reduces the number of test cases
(Without significantly impacting the test coverage)

Crucial concept: Test SOME cases from ALL clusters
(Rather than ALL cases from just SOME clusters)

Example Unit Test Scenario

Let's imagine we are writing a 'Month' class:

```
Month currentMonth = new Month(MARCH,2026);
```

We want to Unit test the 'getDayOfWeek' method:

```
String dayOfWeek = currentMonth.getDayOfWeek(9);
```

You *could* test method with ALL possible integers

```
for(int i=Integer.MIN_VALUE; i<=Integer.MAX_VALUE; i++)
```

But would take a long time (and is unnecessary)
Instead we pick a set of representative numbers...

Suitable Equivalence Partitions

Negative numbers: check invalid inputs are trapped

Zero: a "special" boundary case, in an EP on its own

One: a "special" boundary case, in an EP on its own

Numbers from 2 to 28: should all work the same

Borderlines: 29, 30, 31 (number of days in month)

Big numbers: >31 check invalid inputs are trapped

```
-2    0    1    5    31    32    50
```

Example of Poor Testing

HundredColours

Test cases I would have chosen...

- Both extremes: Black and White
- A selection from the across greyscale range
- Primary light colours (Red, Green, Blue)
- Primary print colours (Cyan, Yellow, Magenta)
- A typical dark colour (low brightness)
- A typical light colour (high brightness)
- A typical dull colour (low saturation)
- A typical vibrant colour (high saturation)
- Different fabrics/materials (cotton, poly, wool)
- Different lighting (sun, halogen, redhead, LED, UV)

But I digress, let us return to software...

Failure Comments

All assertions take a 'failure comment' parameter
A message which gets printed out if assertion fails

These can be extremely useful (if used well)
Try to write specific and detailed messages

Include parameters passed in to provide more detail

```
"Interpreter failed on " + incommingCommand
```

There is a lot of information printed during testing
Careful forethought will make debugging a lot easier

Failure Comment Examples

Bad Failure Comments

```
error claiming cell
wrong exception occurred
incorrect return value
```

Good Failure Comments

```
cell owner still null after attempting to claim cell A2
expected an IOException, but got a NullPointer instead
expected 100 to be returned, but 10 returned instead
```

Note: Failure report includes the test method name
So be sure to give it a clear and descriptive name

Top Ten Tips for Writing Test Cases

1. Keep tests short & simple - test just one feature
   Easier to keep track of what is being tested

2. It's fine to have more that one assertion in a test
   It's good to check feature from different angles

3. OK to have more than one test case in a method
   Clustering for same feature limits method count

4. Don't waste time with brute force coverage
   Use equivalence partitions to target test cases

5. Often need to perform a number of "setup" steps
   Which is where @BeforeEach is useful

Top Ten Tips for Writing Test Cases

6. Give test methods detailed and descriptive names
   So it is clear which tests have failed in the output

7. Include useful and informative failure comments
   So you know what has failed and why it failed

8. Be sure to cover all core feature (obviously)
   But also spend time checking the edge cases

9. Ensure software DOESN'T do what it SHOULDN'T
   As well as it DOES do what it SHOULD

10. Use sub-functions to perform common tasks
    "Divide and conquer" as with normal code

Assessment

Test cases you write will form part of the assessment
So you had better start practicing your testing skills !

We will be checking to see that you cover all features
As well as ensuring error checking and robustness
Also fully check the behaviour of individual methods

REMEMBER:

`A successful test is one that fails (it found a bug)`

You must ensure that your tests actually trap faults
(Not just lists of standard tests for core features)

Marking behaviour in the final assignment

We have accumulated a set of marking test cases
We will use these to assess your final submission

You won't get the opportunity to see OUR test cases
(that would make the assignment far too easy !)

The best way to ensure your code performs well...
Make sure YOUR test cases are as extensive as OURS

Each Year Someone Always Asks...

Why can't we see the actual marking test cases ?

"It's like you aren't giving us the full assignment !"

To help explain why we don't make them available...

Let us consider the DB template example test script:

```
sendCommandToServer("INSERT INTO marks VALUES ('Simon', 65, TRUE);");
sendCommandToServer("INSERT INTO marks VALUES ('Chris', 20, FALSE);");
response = sendCommandToServer("SELECT * FROM marks;");
assertTrue(response.contains("[OK]"), "A valid query was made, but [OK] tag not returned");
assertTrue(response.contains("Simon"), "Simon added to table, but not returned by SELECT *");
assertTrue(response.contains("Chris"), "Chris added to table, but not returned by SELECT *");
response = sendCommandToServer("SELECT * FROM libraryfines;");
assertTrue(response.contains("[ERROR]"), "Access non-existent table, [ERROR] not returned");
```

A Suitable Solution ?

Following code would pass all of those tests just fine:

```
public String handleCommand(String command) {
    if(command.contains("SELECT id")) return "[OK]\n5\n";
    if(command.contains("libraryfines")) return "[ERROR]\n";
    if(command.contains("SELECT *")) return "[OK]\nSimon\nChris\n";
    return "";
}
```

An extreme example, but illustrates a key point...

There is natural reaction to focus on just passing tests

Rather than writing "correct" and "fault-free" code

Test-Driven Development Concerns

As we all know:

The aim of test-driven dev is create the tests FIRST

And THEN write some code that passes those tests

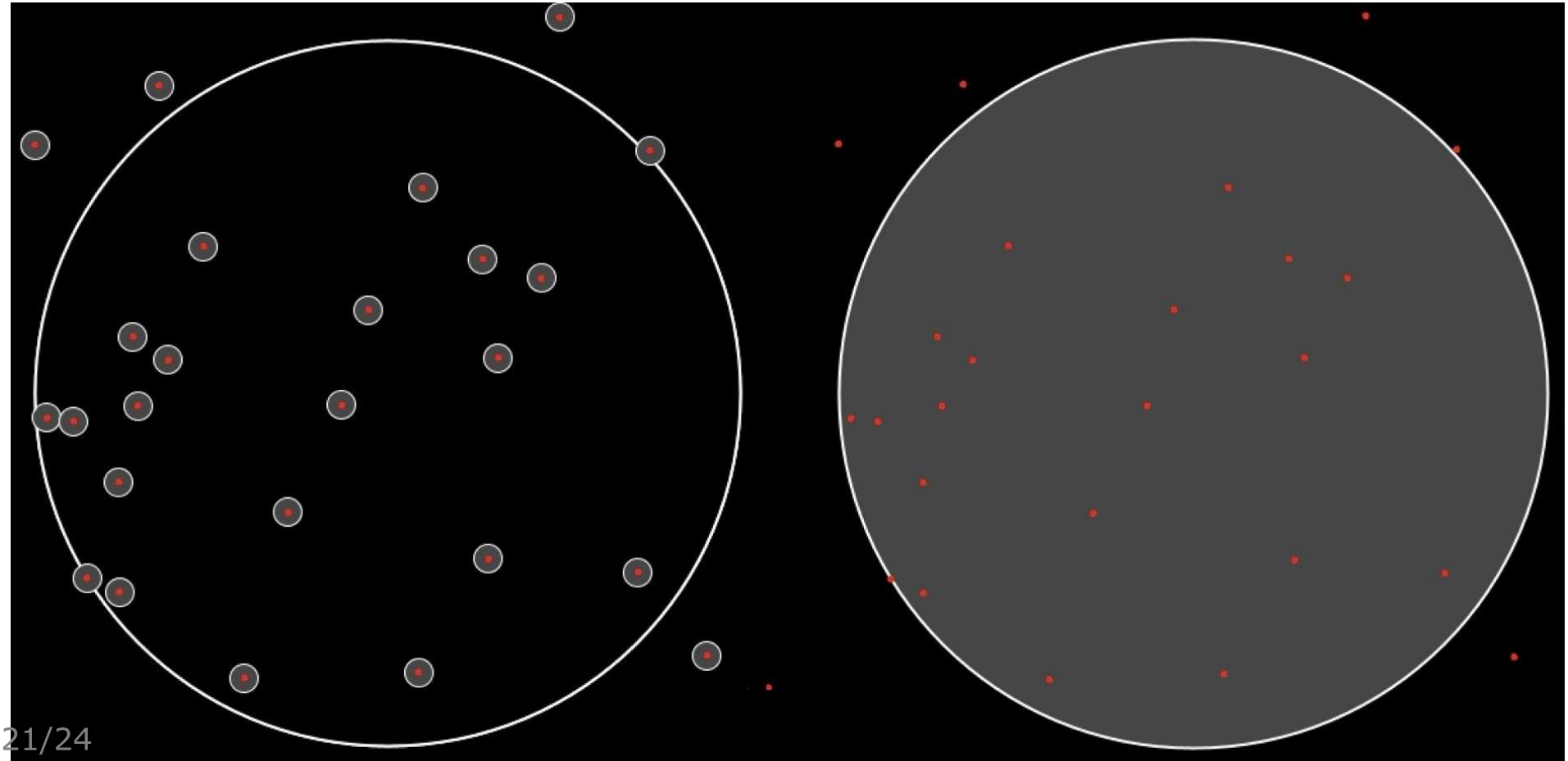
HOWEVER

There is a hidden risk in test-driven development...

TestCaseCoverage

By keep OUR tests under wraps...

We find out which of the below you actually achieved



Simon's Testing Paradox

GOOD code will ALWAYS pass the tests

- but -

Code that passes the tests may not ALWAYS be GOOD

Final Word: Importance of Testing

Writing test cases is a very HARD task to perform
It requires you to really THINK about the situation
You have to actually UNDERSTAND the problem
(Not just mess around poking code until it works)

In many ways analysis & testing are the REAL skills
The skills you should REALLY be trying to acquire
They are a lot more challenging than writing code
(Which is sooooo easy that even AI can do it ;o)